

Authorization Architecture

The permission model, and the registry that is its source of truth

Version	1.8
Status	Final authorization draft — implementation-ready
Audience	Engineering
Companion to	PRD.md

147	292	12	7
ACTIONS	API BINDINGS	PROTECTED CONSTRAINTS	SCOPE VALUES

Contents

- 1. Principles
- 2. Data Model
- 2.1 Permission Policy
- 2.2 Scope
- 2.3 Field Policy
- 2.4 Constraint
- 2.5 Resource Domain
- 2.6 Mandatory Field Policies
- 2.7 Position
- 2.8 User Override
- 2.9 Team
- 3. The Decision Pipeline
- 3.1 Why Segregation of Duties Precedes the Bypass
- 3.2 Service Accounts in the Pipeline
- 3.3 Where the Escape Hatch Is
- 4. Protected Constraints
- 4.1 Absolute — bind every principal
- 4.2 Privileged — bind everyone except Super Admin
- 4.3 Protected Capabilities
- 4.4 Constraint Registration
- 5. Policy Registry and Resource Policies
- 5.1 Shape
- 5.2 Resource Policy Interface
- 5.3 Why a Generic Owner Field Is Insufficient
- 5.4 Example — Project Policy
- 5.4.1 Client Principals Do Not Have a Scope
- 5.4.2 Example — Deal Policy, Two Owner Fields
- 5.5 Scope Resolver
- 6. Action Registry and Route Binding
- 6.1 The Registry Is the Source of Truth, Not the Route
- 6.2 Why Method Is Part of the Key
- 6.3 Enforcement
- 6.4 The Complete Action Registry
- 6.5 API Bindings
- 6.6 CI Checks
- 7. Delegation Evaluation
- 7.1 Approval Delegation — Four Conditions
- 8. Permission Delivery to the Client
- 9. Audit Integration
- 10. Testing Requirements
- 11. Authorization Is Not the Workflow State Machine

TapCRM — Authorization Architecture

Version 1.8 **Status** Final authorization draft — implementation-ready; go-live remains blocked only by [DECISIONS.md](#) BD-2, BD-5, BD-27 and BD-28 **Companion to** [PRD.md](#) **Audience** Engineering

This document is the authorization source of truth. §6.4 contains the complete action registry — all 147 actions with their resource, domain, sensitivity, approval-bearing status, initiator field and grantability. §6.5 binds every one of them to at least one HTTP route. [TECH.md](#) describes how this registry is represented in code — see [TECH.md](#) §6.1 and §7 — but it does not define it, and where the two disagree this document is correct.

This document specifies how an access decision is made. [PRD.md](#) §4 summarizes the model; this is the specification.

1. Principles

#	Principle
1	One entry point, modular policies behind it. Every decision enters through the authorization engine. The engine does not contain per-resource logic; it dispatches to a registered policy.
2	Scope belongs to the action, not the position. The same person legitimately has different reach in different modules.
3	One action resolves to one policy. Never a set of flags where any one suffices, because that leaves the resulting scope ambiguous.
4	Object-level checks everywhere. An endpoint-level check is necessary but never sufficient.
5	Filter before you query. Never fetch broadly and reject rows.
6	Fail closed. An unknown action denies and logs loudly.
7	Segregation of duties outranks privilege. No principal, including the root, approves their own request.
8	Omit, do not null. A field the caller cannot see is absent from the response.
9	Unbounded reach is not a configuration option. There is no all scope. Organization-wide access is a protected capability held by account type.
10	Breadth of subject is not depth of field. Scope decides which records; field policy decides which fields. They are evaluated separately.
11	The action registry, not the URL, is the source of authorization truth.

2. Data Model

2.1 Permission Policy

The unit of authorization. Held on a Position, and optionally overridden per user.

```
PermissionPolicy {
  action      string      // "deals:view" — from PRD Appendix B
  allowed     boolean
  scope       Scope       // see 2.2
  fields      FieldPolicy // optional; default is all readable fields
  constraints  Constraint[] // optional; additional conditions
}
```

2.2 Scope

```
Scope = "none" | "own" | "participant" | "pool" | "team"
       | "department" | "all-people"
```

Scope	Meaning
none	Explicitly denied. Distinct from absent, which means inherit.
own	Resources the principal owns or is the subject of
participant	Resources on which the principal is a named party — the two-sided workflow records: briefs, deliveries, approval requests
pool	Resources owned by members of the principal's agent pool
team	The principal's organizational team, transitively downward through pools and sub-teams beneath it
department	Resources belonging to the principal's department
all-people	Every employee record — people-domain resources only

There is no **all** scope. Organization-wide reach is the `globalAccess` protected capability (§4.4), not a value on a dropdown. A scope enum containing an unbounded option is an invitation to select it.

participant exists so that two-party workflows do not need global reach. The Project Manager and the Development Department Head are the only parties to every Project Brief. Granting each of them a global scope to see their own queue would be a wildly disproportionate grant. **participant** resolves against the resource's named-party fields and gives each of them exactly their queue.

all-people is domain-limited, not merely broad. The engine refuses to evaluate it against a business-domain resource, which is how protected constraint P7 holds structurally rather than by a check that could be forgotten. See §2.5.

2.3 Field Policy

```
FieldPolicy {
  mode      "unrestricted" | "allow" | "deny"
  fields    string[] // path expressions, e.g. "commercial.*"
}
```

unrestricted rather than **all**, so that no reader can mistake a field mode for a scope. There is no scope named **all** (§2.2).

Applied by projection at the serialization boundary. A denied field is omitted from the response object entirely.

2.4 Constraint

```
Constraint {
  type string // "resource-stage" | "self-excluded" | "within-limit" | ...
  params object
}
```

Constraints express conditions the resource must satisfy beyond scope. Example: `deals:view-commercial` for a Sales Agent carries a constraint that the deal was closed by that agent.

2.5 Resource Domain

There are exactly two domains: `people` and `business`. Every resource *instance* resolves to one of them. There is no third value.

A resource policy declares its domain in one of two ways:

```
domain: "people"           // fixed
domain: "business"         // fixed
domain: parentDomainResolver(fn) // derived from the record it attaches to
```

Declaration	Resource types
Fixed <code>people</code>	Employee, Attendance, BreakPolicy, BreakBreach, Shift, LeaveRequest, LeaveBalance, Holiday, Payslip, SalaryStructure, PerformanceRecord, OnboardingWorkflow, BiometricDevice, BiometricPunch, GeofenceEvent
Fixed <code>business</code>	Lead, Callback, Handover, Deal, Approval, Territory, ProjectBrief, Project, Task, Delivery, ChangeRequest, Client, Account, Renewal, BillingTerms, Invoice, CreditNote, Receipt, VendorBill, ExpenseClaim, JournalEntry, LedgerAccount, AccountingPeriod
Derived	Document, Notification, ChatMessage, AuditEntry, Notice

Why derived rather than a third value. A document attached to an employee record is people-domain; the same document type attached to a deal is business-domain. `shared` would name the *declaration style*, not a domain, and the domain is what the engine actually needs at evaluation time. Calling the declaration style a domain would produce policies scoped to `shared`, which is not a thing.

```
DocumentPolicy = {
  resourceType: "document",
  domain: parentDomainResolver(doc => registry
    .get(doc.parentType)
    .resolveDomain(doc.parent)),
  ...
}
```

Rules

#	Rule
RD-1	<code>resolveDomain(instance)</code> returns <code>people</code> or <code>business</code> . It never returns null and never returns a third value.
RD-2	A derived resolver that cannot find its parent fails closed for the authorization evaluation and logs the orphan as a data defect. It does not infer a domain from incomplete data.
RD-3	A derived resolver must terminate. Parent chains are bounded and a cycle is a startup validation failure.

Enforcement. `resolveScope()` raises a **configuration error** — not a permission denial — if `all-people` is evaluated against a `business` resource. This surfaces as a failing test and a startup validation, so an impossible policy cannot be saved in the first place. For derived resources the check runs per instance, at evaluation time, because that is when the domain is known.

```
// Startup validation, for fixed-domain resources
validatePolicy(policy):
  resource = registry.get(actionResource(policy.action))
  if policy.scope == "all-people" and resource.domain == "business":
    throw ConfigurationError(
      `${policy.action} is a business-domain action; all-people is undefined for it`)

// Evaluation time, for derived-domain resources
resolveScope(principal, action, instance):
  if scope == "all-people":
    if registry.get(instance.type).resolveDomain(instance) == "business":
      throw ConfigurationError(...) // logged as a defect, request denied
```

2.6 Mandatory Field Policies

Breadth of subject is not depth of field. A principal holding `all-people` reads every *employee's* record for that module; it does not follow that they read *every field*.

Certain fields carry a **mandatory field policy** that is attached to the resource, not to the position, and that cannot be widened by delegation.

Resource	Restricted fields	Readable by
Payslip	every monetary field	subject; <code>payroll:manage</code> holders
SalaryStructure	every monetary field	subject; <code>payroll:manage</code> holders
Employee	compensation, statutory identifiers, bank details	subject; <code>payroll:manage</code> holders
Employee	permission policies and overrides	<code>access:view</code> holders
LeaveRequest	<code>document</code> attachment	<code>leave:decide</code> holders
PerformanceRecord	review narrative text	subject once published; <code>performance:manage</code> holders
BiometricDevice	device credentials	<code>biometric:manage</code> holders
GeofenceEvent	raw coordinates	<code>identity:manage-geofence</code> holders
Deal	<code>commercials</code>	per protected constraint P1. With BD-9 enabled, <code>creditedValue</code> alone is additionally exposed to the originating agent — nothing else from <code>commercials</code> .
BillingTerms	rate card, discount, credit limit, write-off ceiling	Super Admin; plus <code>billing:view-terms</code> holders where explicitly granted (P8)
Invoice	margin and cost fields	<code>accounting:view-ledger</code> holders. The client and the invoice raiser see the amount charged, never the cost behind it

Rules

#	Rule
MF-1	A mandatory field policy is enforced at projection regardless of the caller's scope. Holding <code>all-people</code> on <code>employee-directory</code> yields no compensation fields.
MF-2	Mandatory field policies are not editable through Access Management. They change by Super Admin action only, and the change is audited.
MF-3	An override cannot widen a mandatory field policy. It can narrow one.
MF-4	<code>payroll:view</code> at <code>all-people</code> grants the payroll module, not payslips. It yields cycle status, run progress and aggregate cost. Reading an individual payslip requires being its subject or holding <code>payroll:manage</code> . This is the practical expression of protected constraint P2, and it is why an HR Executive with directory breadth reads no payslips.

2.7 Position

```
Position {
  code          string    // immutable
  name          string
  organizationalLevel int    // 1..99; NOT an authorization grant
  departmentRef  ref
  parentPosition ref
  policies       PermissionPolicy[]
  approvalLimits { dealValueMax, discountPercentMax,
                  allowCustomTerms, requiresContractSignoff }
  isSystem       boolean
  status         "active" | "inactive"
}
```

There is no position-level `scope` field, and no `teamScoped` flag. Whether a policy is team-bounded is stated by that policy's own scope. A separate boolean would be a second source of truth for the same fact.

2.8 User Override

An override is a complete policy, not a boolean.

```
UserOverride {
  action      string
  allowed     boolean
  scope       Scope      // may differ from the position default
  fields      FieldPolicy // optional
  constraints  Constraint[] // optional
  reason      string     // REQUIRED
  grantedBy   ref User
  grantedAt   datetime
  expiresAt   datetime?   // evaluated at AUTHORIZATION TIME; the nightly
                          // job performs cleanup only – see OV-1
}
```

Rules

#	Rule
OV-1	Expiry is evaluated on every authorization call , not by a scheduled job. <code>resolve()</code> skips any override whose <code>expiresAt <= now</code> , so an override expiring at 10:00 stops applying at 10:00, not at the next cleanup run. A permission that outlives its expiry by up to 24 hours is not an expiring permission.
OV-2	The nightly job is responsible for cleanup only : marking expired rows, invalidating cached policy sets, writing the audit entry, and notifying both parties. It never determines whether a permission is currently valid.
OV-3	Cached permission sets (§8) carry the earliest <code>expiresAt</code> among the principal's overrides as a cache deadline, so a cached decision can never outlive an expiry either.
OV-4	An override with <code>expiresAt</code> in the past is treated as absent, not as a denial. Absent means inherit the position's policy.

Merge rule. For a given action, an override **replaces** the position's policy entirely. It does not merge field-by-field. Partial merges produce combinations nobody configured and nobody can predict.

2.9 Team

```
Team {
  code, name
  kind      "sales-team" | "sales-pool" | "dev-subteam"
  departmentRef ref
  leadUser   ref User
  parentTeam ref Team?    // pool → sales-team
  sharedVisibility boolean // default false
  status
}
```

`User.teamRef` is organizational ownership. Project membership is a separate relationship on the project. Conflating them is explicitly prohibited by `PRD.md` rule T-3.

3. The Decision Pipeline

Evaluated in this exact order. The order is the specification — in particular, segregation of duties precedes the privilege bypass.

1. AUTHENTICATION AND ACCOUNT STATE
 - Valid token? Session version current? Account active?
 - Fail → 401
2. SEGREGATION OF DUTIES
 - Is this an approval-bearing action on a record this principal raised?
 - Fail → 403, logged, ALWAYS — no principal is exempt
3. ABSOLUTE CONSTRAINTS
 - A2 client isolation, A3 record immutability
 - Fail → 403 (or 404 for A2), ALWAYS — no principal is exempt
4. SUPER ADMIN BYPASS
 - accountType === "super-admin" → allow, audited
 - (Steps 2 and 3 have already run and cannot be reached again)
5. PRIVILEGED CONSTRAINTS
 - Pl..P8 — apply to every principal who reaches this step; Super Admin is bypassed only because the protected constraint is bypassed
 - Fail → 403, or field omission where the constraint is field-level
6. POLICY RESOLUTION
 - resolve(user, action) → PermissionPolicy | null
 - null or allowed === false → 403, logged
 - Unknown action → 403, logged as a defect
7. SCOPE EVALUATION
 - Single record → resourcePolicy.check(user, action, resource, scope)
 - List → resourcePolicy.filter(user, action, scope)
 - Fail → 403 for a single record; empty-matching filter for a list
8. CONSTRAINT EVALUATION
 - Each constraint on the policy is evaluated against the resource
 - Fail → 403
9. FIELD PROJECTION
 - Apply the policy's FieldPolicy; omit denied paths
10. AUDIT
 - Sensitive reads and all writes recorded

3.1 Why Segregation of Duties Precedes the Bypass

A system sold to companies must be able to demonstrate that no single account can originate and approve the same transaction. If the root principal can do both, the control does not exist — it is a convention that happens to be followed.

Placing steps 2 and 3 first costs one predicate evaluation and makes the control provable. Super Admin retains every other power, including the power to reassign an approval to someone else, which is the legitimate way to unblock a request nobody else can decide.

3.2 Service Accounts in the Pipeline

A service account has no position, team or reporting line, so steps 6 and 7 resolve differently.

```
Step 6 POLICY RESOLUTION
  action ∈ serviceAccount.allowedActions? → allow
  otherwise → 403
  Position policies are NEVER consulted.

Step 7 SCOPE EVALUATION
  organizationId == serviceAccount.organizationId ← ALWAYS FIRST
  AND resourceType ∈ serviceAccount.allowedResources
  AND resource matches serviceAccount.recordFilter (if set)
  otherwise → 403
```

Tenancy is not part of `recordFilter`. `organizationId` is injected by the data-access layer from the credential, before `recordFilter` is applied, and `recordFilter` cannot reference or override it. Were tenancy expressed as a filter, an empty `recordFilter` — the natural default for "no extra narrowing" — would mean no tenant boundary at all. The narrowing filter and the isolation boundary must not share a mechanism.

Steps 1 through 5 run unchanged. In particular a service account is bound by every absolute constraint: it cannot approve a request it raised, cannot cross a client boundary, and cannot mutate an immutable record. Additional gates apply at step 1: credential expiry, IP allowlist and rate limit.

Service accounts can never hold `globalAccess`, `access:delegate`, `users:manage`, `payroll:manage` or `notepad:view-all`, and this is enforced at credential creation, not at request time.

3.3 Where the Escape Hatch Is

If a request has no eligible approver other than its requester, it **escalates** automatically to the next level up the reporting chain (`PRD.md` AP-5 / SD-3). If that resolves to nobody — the Super Admin raised it — the request routes to a configured secondary approver nominated at setup. An organization with exactly one administrator must nominate a second person for approval-bearing workflows; that is a governance requirement, not a software limitation.

4. Protected Constraints

Enforced by the engine, not by policy configuration. Access Management renders protected capabilities as locked with the governing rule named.

4.1 Absolute — bind every principal

#	Constraint	Enforcement
A1	Segregation of duties: no principal approves, confirms or signs off an item they initiated	Step 2; via <code>initiatorField(action)</code> on the resource policy — see §4.1.1
A2	Client isolation: a client principal reaches only their own account's resources	Step 3; account filter injected before query; violations return 404
A3	Record immutability: published payslips, recorded approvals, audit entries, delivery sign-offs, issued invoices, recorded receipts and posted journal entries cannot be edited	Step 3; write rejected at the persistence boundary, not only in the handler
A4	Financial period integrity: nothing posts into a closed accounting period. Super Admin may reopen a period with a recorded reason, but may not post into a closed one	Step 3; predicate on the posting date against the period register

4.1.1 Resolving the Initiator

There is no universal `raisedBy` field, and assuming one would silently disable the control on every resource that names its initiator differently — which is most of them.

The segregation engine asks the resource policy:

```
SoDPolicy.check(principal, action, resource):
    field = registry.get(resource.type).initiatorField(action)
    if (field === null) return PASS // action is not approval-bearing
    if (field === UNRESOLVED) return DENY // fail closed, log as a defect
    return resolvePath(resource, field) === principal.id ? DENY : PASS
```

Action	Initiator field	Rationale
<code>approvals:decide</code>	<code>requestedBy</code>	The requester never decides
<code>deals:approve</code>	<code>createdBy</code>	Whoever opened the deal does not approve it
<code>deals:confirm-payment</code>	<code>payment.initiatedBy</code>	Whoever recorded the payment does not confirm it
<code>deals:record-win</code>	<code>closedBy</code>	Distinct from the approver and the payment confirmer
<code>handoff:confirm</code>	<code>createdBy</code>	The Sales Head who drafted the brief does not confirm it
<code>handoff:review</code>	<code>confirmedBy</code>	The PM who confirmed the brief does not pass it at feasibility
<code>delivery:signoff</code>	<code>deliveredBy</code>	Whoever delivered does not sign off
<code>changes:classify</code>	<code>raisedBy</code>	Whoever raised the change does not price it
<code>leave:decide</code>	<code>requestedBy</code>	Nobody approves their own leave
<code>attendance:correct</code>	<code>requestedBy</code>	Nobody approves their own correction
<code>breaks:review-breach</code>	<code>userId</code>	Nobody reviews their own breach
<code>shifts:approve</code>	<code>requestedBy</code>	Nobody approves their own shift change
<code>tasks:review</code>	<code>assignedTo</code>	Nobody signs off their own submission
<code>performance:manage</code>	<code>subjectId</code>	Nobody writes their own review
<code>access:decide-role-change</code>	<code>requestedBy</code>	Nobody approves their own promotion

Action	Initiator field	Rationale
<code>invoicing:issue</code>	<code>createdBy</code>	Where invoice approval is configured, the raiser does not approve
<code>payments:reconcile</code>	<code>recordedBy</code>	Whoever recorded a receipt does not reconcile it
<code>payments:refund</code>	<code>requestedBy</code>	Whoever raises a refund does not approve it
<code>receivables:write-off</code>	<code>proposedBy</code>	Whoever proposes a write-off does not approve it
<code>payables:approve-bill</code>	<code>createdBy</code>	Whoever entered a bill does not approve it
<code>payables:approve-claim</code>	<code>claimedBy</code>	Nobody approves their own expense claim
<code>payables:execute-run</code>	<code>approvedBy</code>	Whoever approved the bills does not execute the payment
<code>accounting:close-period</code>	<code>lastPostedBy</code>	Whoever posted does not close the period

Rules

#	Rule
SD-A	Every approval-bearing action must declare an initiator field. A registry validation at startup fails the build if an action marked approval-bearing returns <code>null</code> .
SD-B	<code>UNRESOLVED</code> — a declared field that does not exist on the record — denies . A segregation control that silently no-ops when the schema drifts is worse than no control, because it is believed.
SD-C	Where two distinct roles must be separated on the same record — initiating a payment and confirming it — they are two actions with two initiator fields, not one action with a compound rule.
SD-D	Adding an approval-bearing action requires adding its initiator field in the same change. There is no default.

4.2 Privileged — bind everyone except Super Admin

#	Constraint	Enforcement
P1	Closing-terms confidentiality: an agent cannot read <code>commercials</code> of a deal beyond their own approval limit that they did not close. Scoped to the Deal resource only — account ownership is governed by the account policy and never widens this	Step 5, field-level; <code>commercials</code> omitted
P2	Payslip privacy: readable by subject and payroll holders only; team scope never reaches payslips	Step 5; scope coerced to <code>own</code> unless the policy is <code>payroll:manage</code>
P3	Notepad privacy: owner and Super Admin only	Step 5; no policy grants <code>notepad:view-all</code> except to Super Admin
P4	Leave document privacy: attachments readable by HR only	Step 5, field-level; <code>document</code> omitted for the acknowledging manager
P5	Task-level client data: base delivery employees see no client contact, payment or contract fields	Step 5, field-level projection on project and task responses
P6	Cross-sub-team isolation: a sub-team manager cannot reach another sub-team's work	Step 5; team resolution refuses to traverse sibling <code>dev-subteam</code> units
P8	Client billing terms — rate card, negotiated pricing, discount, payment terms, credit limit, write-off ceiling — are written by Super Admin alone . Never grantable, never delegable, and not transferred to a Finance Manager. Read access is separately grantable and off by default	Step 5; <code>billing:set-terms</code> has <code>positionGrantable = false</code>
P7	HR domain boundary: an HR principal cannot read business-domain records — leads, deals, deal commercials, revenue, project profitability, client financial records, projects or tasks — at any scope. Compensation and payroll are people-domain resources governed by their own policies and mandatory field policies (§2.6), and are not covered by this constraint	Step 5; <code>all-people</code> is undefined for business-domain actions

4.3 Protected Capabilities

Distinct from protected *constraints*. A constraint restricts what a granted capability can reach; a protected capability cannot be granted through Access Management at all.

Capability	<code>positionGrantable</code>	<code>delegationAllowed</code>	<code>superAdminOnly</code>
<code>globalAccess</code>	n/a — derived from account type, never an action	—	—
<code>notepad:view-all</code>	false	false	true
<code>access:delegate</code>	true	false	true
<code>users:manage</code>	true	false	true
<code>payroll:manage</code>	true	false	true
<code>system:manage-settings</code>	true	false	true
<code>system:manage-thresholds</code>	true	false	true
<code>system:manage-integrations</code>	true	false	true
<code>system:manage-retention</code>	true	false	true
<code>audit:manage-holds</code>	true	false	true
<code>audit:export</code>	true	false	true
<code>billing:set-terms</code>	false	false	true
<code>accounting:reopen-period</code>	true	false	true
<code>accounting:close-period</code>	true	false	false
<code>receivables:write-off</code>	true	true	false
Mandatory field policies	not an action — changed by Super Admin only (MF-2)	—	—

`globalAccess` is a function, not a field.

```
globalAccess(principal) = principal.accountType === "super-admin"
```

That is the entire definition, and it is the only definition. Specifically there is:

- no `user.globalAccess` column
- no Position policy that can carry it
- no `UserOverride` that can grant it
- no delegation path that can confer it
- no Access Management control that renders it
- no `Scope` value that expresses it

Why a function rather than a flag. A boolean on a record is a boolean somebody can set. The moment `globalAccess` exists as stored state, it acquires a write path, and every write path eventually acquires a caller. Deriving it from account type means granting it requires changing what kind of principal someone is — a far louder act than editing a permission — and it cannot be done by accident from an administration screen.

Implementation rule. `globalAccess` is evaluated at step 4 as part of the account-type bypass. It is never resolved from a policy list, because a capability that can be looked up in a list can be written into one.

4.4 Constraint Registration

Constraints are registered declaratively, not written inline at call sites:

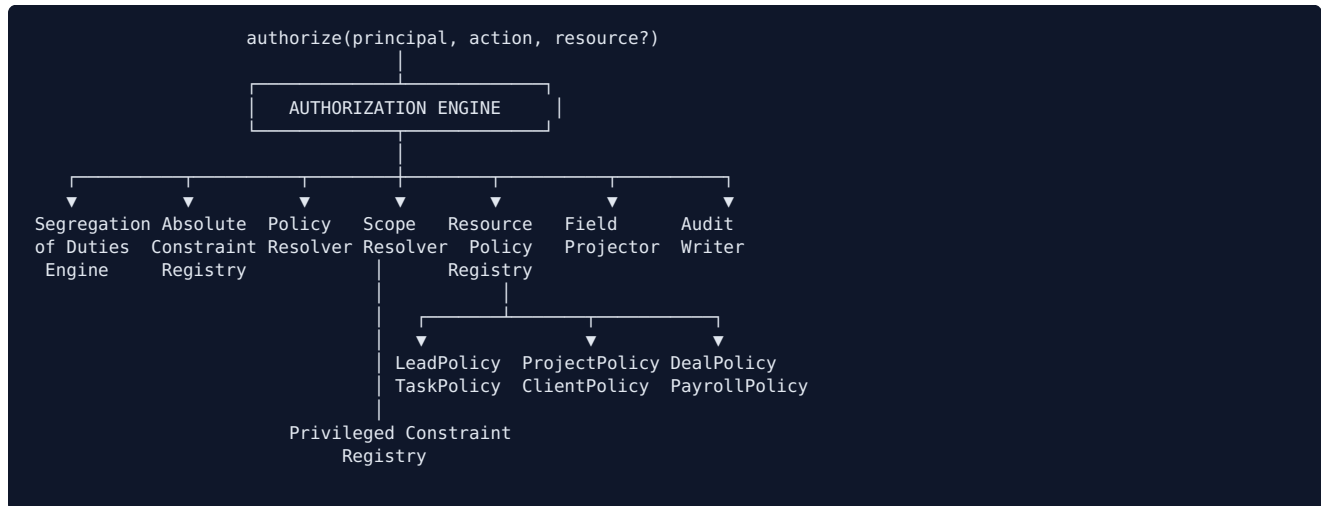
```
registerConstraint({
  id: "P2",
  kind: "privileged",
  appliesTo: ["payroll:view"],
  evaluate: (principal, action, resource, policy) => {
    if (policyGrants(principal, "payroll:manage")) return PASS;
    if (resource.subjectId === principal.id) return PASS;
    return DENY("P2: payslips are readable by their subject and payroll only");
  }
});
```

A new constraint is added to the registry, not to a handler. This is what stops the constraint list becoming an ever-growing sequence of special cases scattered through the codebase.

5. Policy Registry and Resource Policies

5.1 Shape

One entry point, modular policies behind it.



Everything in that diagram is authorization. Segregation of duties, absolute constraints, privileged constraints and field policies are authorization decisions and live inside the engine — not in controllers. The temptation to treat P1 or P7 as "business policy, not a permission" and implement it in a handler is exactly how a rule ends up duplicated across three call sites and enforced in two of them.

The engine never contains `if (resourceType === "lead")`. It looks up the registered resource policy and delegates.

5.2 Resource Policy Interface

One interface, one definition. Domain (§2.5) is part of it, not a separate declaration.

```

ResourcePolicy {
  resourceType : string
  domain      : "people" | "business" | ParentResolver // §2.5

  // Object-level: may this principal perform this action on this record?
  check(principal, action, resource, scope) -> ALLOW | DENY(reason)

  // List-level: the query filter expressing this scope for this resource
  filter(principal, action, scope) -> QueryFragment

  // Which fields of this record name a party to it, per action.
  // Required for `participant` scope. Returns [] where the concept
  // does not apply, which makes `participant` resolve to MATCH_NOTHING.
  participantFields(action) -> string[]

  // Which field names the principal who initiated the item, per action.
  // Required by segregation of duties (§4.1 A1). Returns null where the
  // action is not approval-bearing.
  initiatorField(action) -> string | null
}
  
```

Invariant. `filter` and `check` must agree. A record returned by `filter` must pass `check`, and vice versa. Verified by a property test per resource policy: generate records, run both, assert agreement.

5.3 Why a Generic Owner Field Is Insufficient

Resources have different ownership shapes:

Resource	Ownership	Participants	Initiator (by action)
Lead	one owning agent	—	—
Callback	assignee, plus the lead's owner	—	—
Deal	two owner fields: <code>originatingAgent</code> and <code>closedBy</code> — a deal may be sourced in one team and closed in another (<code>PRD.md</code> DL-12)	—	<code>deals:approve</code> → <code>createdBy</code> ; <code>deals:confirm-payment</code> → <code>payment.initiatedBy</code> ; <code>deals:record-win</code> → <code>closedBy</code>
ProjectBrief	drafter, confirmer, reviewer — three parties	<code>createdBy</code> , <code>confirmedBy</code> , <code>reviewedBy</code>	<code>handoff:confirm</code> → <code>createdBy</code> ; <code>handoff:review</code> → <code>confirmedBy</code>
Project	project manager, delivery owner, client, assigned team	—	—
Task	assignee, assigner, sub-team, and the project's Project Manager via the dotted line (<code>PRD.md</code> §3.7.1)	—	<code>tasks:review</code> → <code>assignedTo</code>
Delivery	deliverer, sharer, signer	<code>deliveredBy</code> , <code>sharedBy</code> , <code>signedOffBy</code>	<code>delivery:signoff</code> → <code>deliveredBy</code>
ChangeRequest	raiser, classifier, assignee	<code>raisedBy</code> , <code>classifiedBy</code>	<code>changes:classify</code> → <code>raisedBy</code>
Approval	requester and eligible approvers	<code>requestedBy</code> , <code>approverCandidates</code>	<code>approvals:decide</code> → <code>requestedBy</code>
Handover	initiator and target	<code>fromUser</code> , <code>toUser</code>	—
LeaveRequest	subject	—	<code>leave:decide</code> → <code>requestedBy</code>
AttendanceCorrection	requester and approver	—	<code>attendance:correct</code> → <code>requestedBy</code>
BreakBreach	subject and reviewer	—	<code>breaks:review-breach</code> → <code>userId</code>
Client	account owner, plus every project's team	—	—
Invoice	client, raiser, issuer	—	<code>invoicing:issue</code> → <code>createdBy</code>
Receipt	client, recorder	—	<code>payments:reconcile</code> → <code>recordedBy</code>
VendorBill	vendor, creator, approver	—	<code>payables:approve-bill</code> → <code>createdBy</code>
ExpenseClaim	claimant, approver	<code>claimedBy</code> , <code>approvedBy</code>	<code>payables:approve-claim</code> → <code>claimedBy</code>
AccountingPeriod	poster, closer	—	<code>accounting:close-period</code> → <code>lastPostedBy</code>
Document	inherits its parent record entirely	inherits	inherits

A single `ownerField` parameter cannot express "a developer sees this project because they are in `assignedTo` , and the Project Manager sees it because they are `projectManager` , and the Development Department Head sees it because it belongs to their department." Each resource policy encodes its own shape.

5.4 Example — Project Policy

```
ProjectPolicy = {
  resourceType: "project",
  domain: "business",

  filter(principal, "projects:view", scope):

    // Client principals never enter the scope switch. Client isolation is a
    // principal-to-resource relationship, not an organizational scope, and is
    // applied at step 3 of the pipeline as absolute constraint A2.
    // See 5.4.1.

    switch (scope):
      "department" → { departmentRef: principal.departmentRef }
      "team"       → { assignedTeams: { in: teamIds(principal) } }
      "participant" → { or: participantFields("projects:view")
                        .map(f => ({ [f]: principal.id })) }
      "own"        → { or: [ { assignedTo: principal.id },
                            { projectManager: principal.id },
                            { deliveryOwner: principal.id } ] }
      // A Project Manager's "own" is their portfolio. TaskPolicy resolves
      // "own" for a PM by joining through the task's project to its
      // projectManager — the dotted line, not a field on the task.
      default      → MATCH_NOTHING,

  participantFields(action):
    return ["projectManager", "deliveryOwner"],

  initiatorField(action):
    return null // projects are not approval-bearing
}
```

`MATCH_NOTHING` is a filter that provably matches no document — never an empty filter, which would match everything.

There is no **"all"** case and no **"global"** case in any resource policy. Organization-wide reach never reaches step 7. It is resolved at step 4 of the pipeline, where the account-type bypass short-circuits before any resource policy is consulted. A resource policy that handled a global scope would be a second place global access could be granted.

5.4.1 Client Principals Do Not Have a Scope

`account` is **not** a scope value and does not appear in the `Scope` enum. Client isolation is a relationship between a principal and a resource, evaluated as absolute constraint A2 at step 3, before policy resolution.

```
// Step 3, before any scope evaluation
if (principal.accountType === "client") {
  const accessible = accessibleResourceFilter(principal, resourceType)
  if (accessible === UNSUPPORTED) return DENY_404 // no client access at all
  injectFilter(accessible)                       // narrows the query
  // evaluation continues; the client's own policy set still applies
}
```

Modelling this as a scope would put an outside party's boundary inside the same enum an administrator edits, which is exactly where it should not be. The practical consequence is that a misconfigured position can never accidentally select the value that lets an employee see the world the way a client does, or vice versa.

5.4.2 Example — Deal Policy, Two Owner Fields

Deal visibility is the clearest case for why a generic owner field fails. Because handover is open across teams, a deal has **two** owners and a principal sees it through either.

```
DealPolicy = {
  resourceType: "deal",
  domain: "business",

  filter(principal, "deals:view", scope):
    reach = reachableUserIds(principal, scope) // own | pool | team | department
    if (reach === MATCH_NOTHING) return MATCH_NOTHING
    return { or: [ { originatingAgent: { in: reach } },
                  { closedBy: { in: reach } } ] },

  check(principal, "deals:view", deal, scope):
    reach = reachableUserIds(principal, scope)
    return reach.has(deal.originatingAgent) || reach.has(deal.closedBy)
      ? ALLOW : DENY("outside both attribution chains"),

  participantFields(action): return ["originatingAgent", "closedBy"],
  initiatorField(action): return {
    "deals:approve": "createdBy",
    "deals:confirm-payment": "payment.initiatedBy",
    "deals:record-win": "closedBy"
  }[action] ?? null
}
```

Why a union rather than an intersection. Both chains contributed: one sourced the customer, the other closed them. A supervisor who closed a deal must see the deal they closed; the agent's team lead must see a deal their agent sourced. Requiring both would hide the deal from everyone in a cross-team case, which is the case the rule exists for.

Why this is not a leak. `reachableUserIds` is still bounded by the principal's scope. A Supervisor with `pool` scope sees the deal only if one of the two attributed users is in their own pool. Peer supervisors in other pools see nothing, which is exactly the §3.5.1 outcome.

Protected constraint P1 still applies on top: an agent who sourced but did not close sees the deal without its `commercials`.

5.5 Scope Resolver

Shared by every resource policy. Computes, once per request and cached:

Function	Returns
<code>subordinateIds(principal)</code>	Transitive closure of <code>reportsTo</code> beneath the principal
<code>teamIds(principal)</code>	The principal's team plus every team beneath it, refusing to traverse sibling sub-teams (P6)
<code>poolIds(principal)</code>	Pools the principal leads
<code>departmentId(principal)</code>	The principal's department
<code>participantMatch(principal, policy, action)</code>	Delegates to <code>policy.participantFields(action)</code> and builds a disjunction over those fields. Returns <code>MATCH_NOTHING</code> when the list is empty.

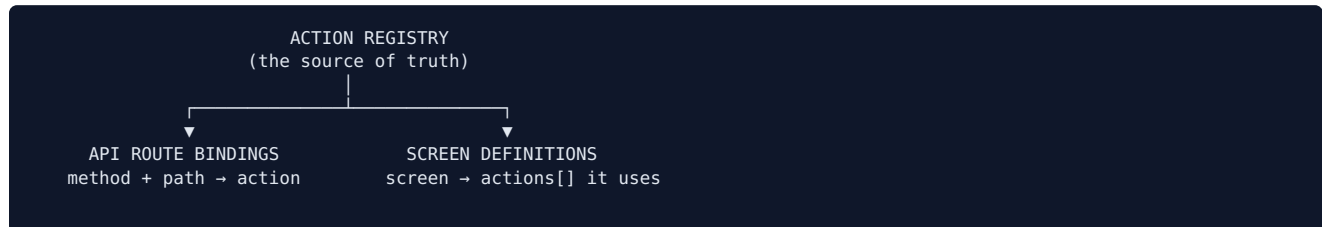
`participant` is deliberately the one scope the resolver cannot compute alone. Who counts as a party depends entirely on the resource, so the resolver asks the resource policy rather than guessing. A policy that returns an empty `participantFields` list makes `participant` scope resolve to nothing, which fails closed.

Computed once per request, not once per record. This is what keeps step 7 inside the 20 ms budget in `PRD.md` NF-5.

6. Action Registry and Route Binding

6.1 The Registry Is the Source of Truth, Not the Route

A URL is not an authorization concept. One screen exercises many actions, and one URL family carries different capabilities by method. The **action registry** is the single source; both route tables reference it.



Action registry entry

```

ActionDefinition {
  action      "deals:approve"
  module      "deals"
  resource     "deal"           // which ResourcePolicy governs; null if none
  domain      "business"       // from §2.5, via the resource
  description  "Decide a deal within the holder's approval limits"
  sensitive    true            // audited on use
  approvalBearing true        // if true, initiatorField() is mandatory (§4.1.1)

  grantPolicy {
    positionGrantable boolean // may appear in a Position's policy list
    delegationAllowed  boolean // may be granted via access:delegate
    superAdminOnly     boolean // only Super Admin may grant it
  }
}
  
```

Three independent booleans, not one enum. Grantability has three separate questions and an enum forces them into one axis, which loses information the moment an action is position-grantable but not delegable.

Question	Field
Can this appear in a Position's policy list at all?	<code>positionGrantable</code>
Can a delegate grant it to a subordinate?	<code>delegationAllowed</code>
Is Super Admin the only principal who may grant it?	<code>superAdminOnly</code>

Worked examples

Action	positionGrantable	delegationAllowed	superAdminOnly	Meaning
<code>deals:view</code>	true	true	false	Ordinary. Any position may hold it; a delegate may grant it within the ceiling.
<code>deals:approve</code>	true	true	false	Ordinary, but bounded additionally by approval limits.
<code>users:manage</code>	true	false	true	A position may hold it, but only Super Admin can put it there.
<code>access:delegate</code>	true	false	true	Same. A delegate cannot create another delegate.
<code>notepad:view-all</code>	false	false	true	Cannot appear in any Position policy. Super Admin only, by account type.
	true	false	true	

Action	positionGrantable	delegationAllowed	superAdminOnly	Meaning
<code>system:manage-thresholds</code>				Configurable position, Super Admin grants it.
<code>payroll:manage</code>	true	false	true	Position may hold it; Super Admin grants it.

Validation rules

#	Rule
GP-1	<code>positionGrantable = false</code> implies <code>delegationAllowed = false</code> . An action that cannot sit in a policy list cannot be delegated into one. Violation fails the build.
GP-2	<code>superAdminOnly = true</code> implies <code>delegationAllowed = false</code> . Violation fails the build.
GP-3	<code>positionGrantable = false</code> means the action cannot appear in a Position policy. Today the only such actions are <code>notepad:view-all</code> and <code>billing:set-terms</code> ; <code>globalAccess</code> is a separate account-type capability and is not an action. Adding another non-position-grantable action requires explicit review.
GP-4	The Access Management UI renders an action as editable only when <code>positionGrantable</code> is true, and as editable <i>by the current actor</i> only when <code>superAdminOnly</code> is false or the actor is Super Admin.
GP-5	<code>approvalBearing = true</code> requires the governing resource policy to return a non-null <code>initiatorField</code> for this action. Startup validation, not a runtime check.

API route binding

```
ApiBinding {
  method      "POST"
  path        "/api/deals/:id/approve"
  action      "deals:approve" // MUST exist in the registry
  resourceParam "id"          // required when the action names a resource
}
```

Screen definition

```
ScreenDefinition {
  screen      "/sales/deals/:id"
  module      "deals"
  label       "Deal detail"
  icon        "briefcase"
  navGroup    "sales"
  requires    ["deals:view"] // to reach the screen
  uses        ["deals:edit", "deals:approve",
               "deals:view-commercial",
               "deals:confirm-payment"] // to render controls
}
```

`requires` gates navigation. `uses` drives which controls render. Neither protects data — that is the API binding's job, evaluated server-side on every request.

6.2 Why Method Is Part of the Key

Authorization cannot be inferred from a path alone:

Method	Path	Action
GET	<code>/api/deals/:id</code>	<code>deals:view</code>
PATCH	<code>/api/deals/:id</code>	<code>deals:edit</code>
POST	<code>/api/deals/:id/approve</code>	<code>deals:approve</code>
POST	<code>/api/deals/:id/close</code>	<code>deals:record-win</code>

Same URL family, four different capabilities. The manifest is keyed on method plus path template.

6.3 Enforcement

#	Rule
RM-1	An API route registered at boot with no binding causes a startup failure , not a runtime 403. A missing binding is a defect and defects should surface at deploy time.
RM-2	A binding naming an action absent from the registry fails the build.
RM-3	A binding whose action names a resource must declare <code>resourceParam</code> , so the object-level check has something to load.
RM-4	A screen listing an action absent from the registry fails the build.
RM-5	Every action in the registry must be reachable by at least one API binding. An unreachable action is dead configuration and fails the build.
RM-6	The sidebar is generated by filtering screens through the principal's resolved policy set using <code>requires</code> .
RM-7	Client-side filtering is presentation only. Removing a link is never the mechanism that protects data.
RM-8	The registry, the bindings and the screens are the three artefacts the permission matrix in <code>PRD.md</code> §6 is verified against.

6.4 The Complete Action Registry

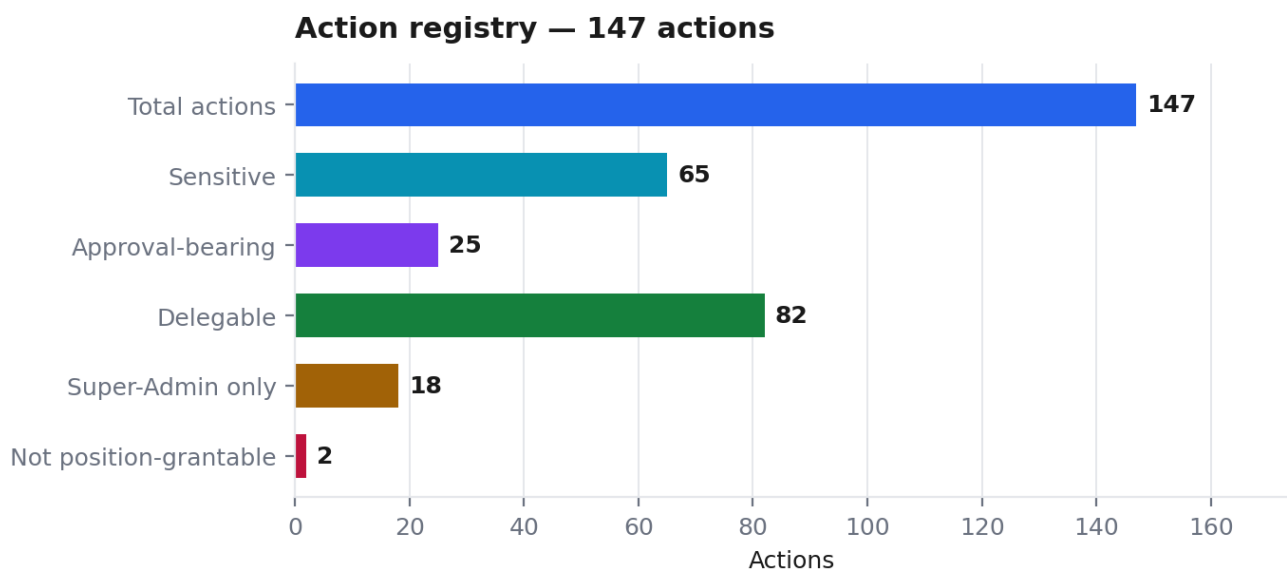


Figure 1 — Registry totals. 65 of 147 actions are sensitive and audited on every use, not only on grant.

Actions by resource domain

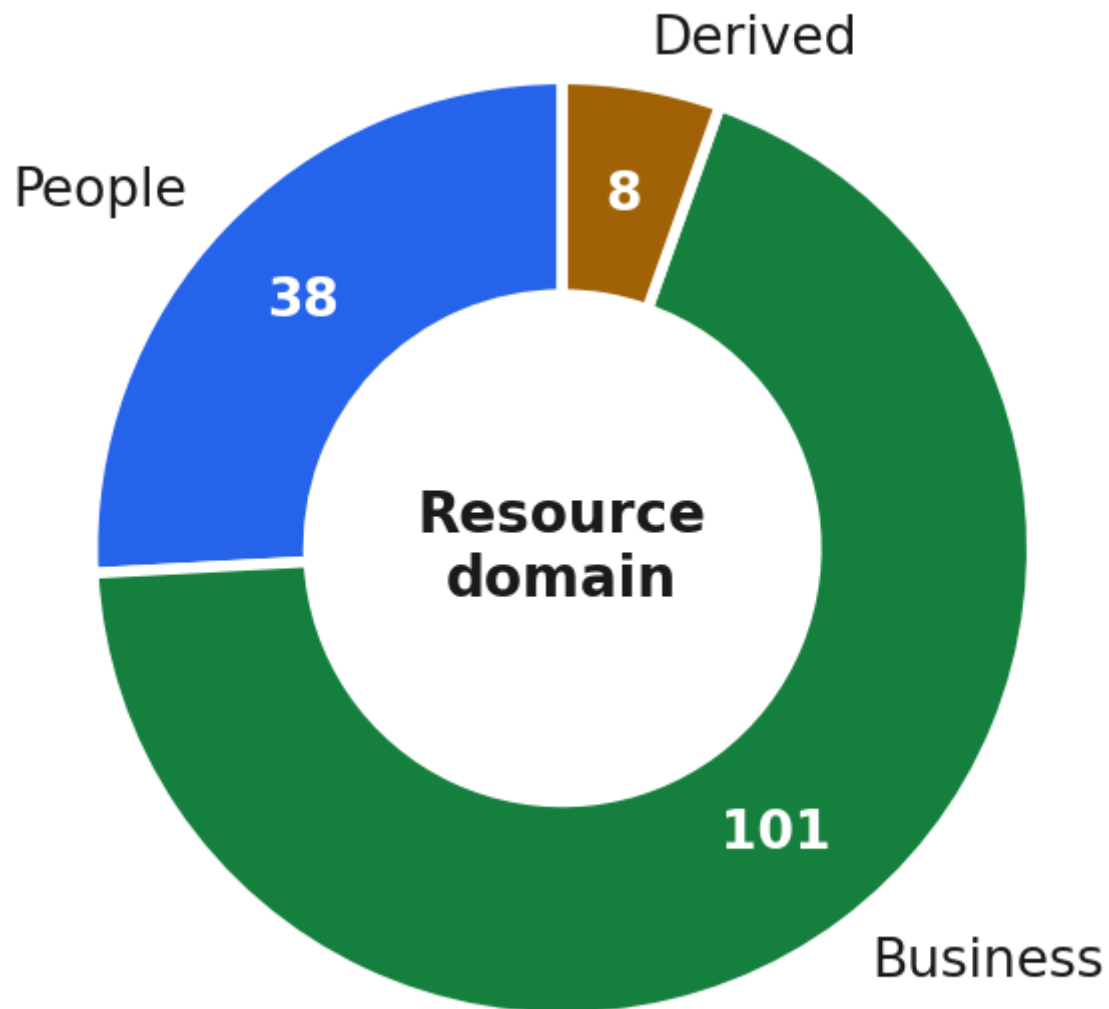


Figure 2 — Actions by resource domain. The all-people scope is undefined for business-domain actions, which is how protected constraint P7 holds structurally.

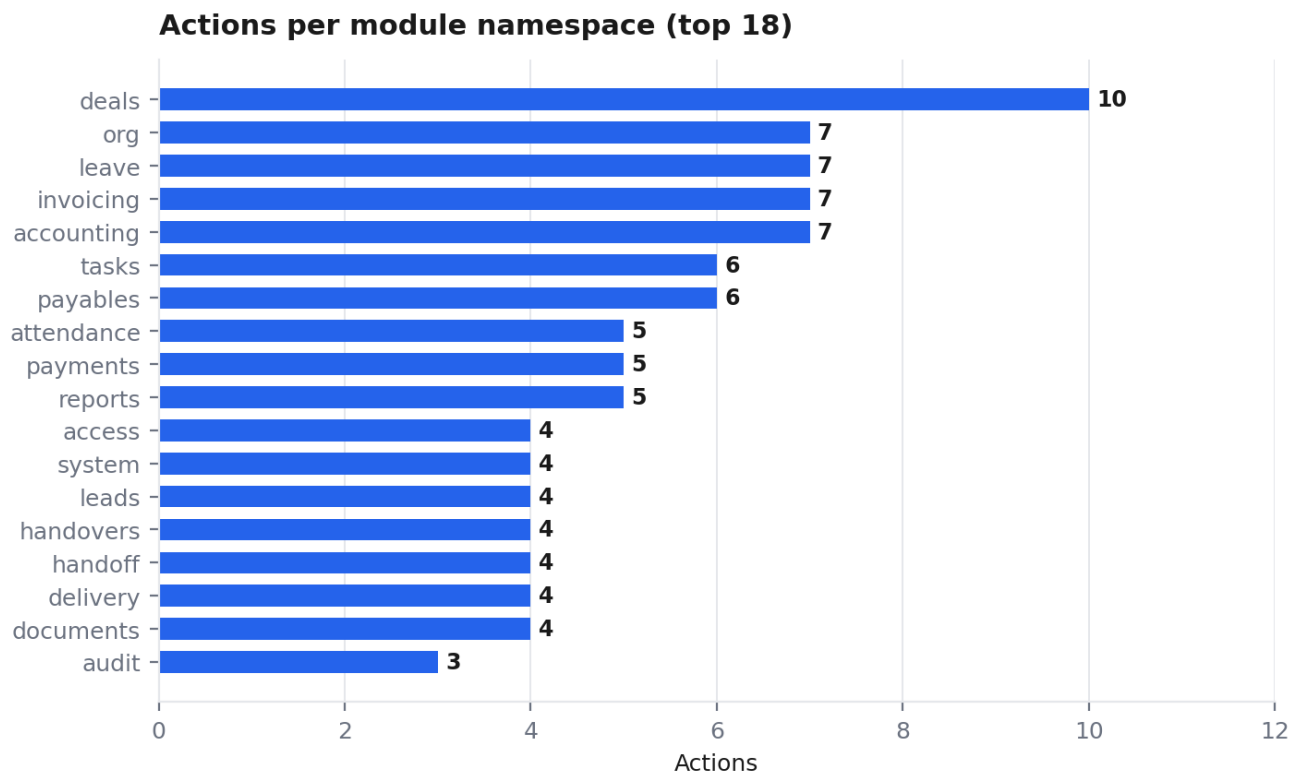


Figure 3 — Actions per module namespace.

This is the registry. Every action in the product, with its full definition. [PRD.md](#) Appendix B is the human-readable catalogue of what each action *means*; this table is the machine-readable definition of how each is *evaluated*. The two must agree, and a CI check asserts it (§6.6 RM-9).

147 actions. Column meanings:

Column	Meaning
Resource	Which <code>ResourcePolicy</code> governs object-level checks and list filtering. <code>-</code> means the action operates on no object — configuration or a derived report — and is gated by policy alone.
Domain	<code>people</code> <code>business</code> . A derived resource resolves to one of these two domains at evaluation time; <code>derived</code> is a policy declaration style, not a third domain.
Sens	Sensitive: every <i>use</i> is written to the access audit, not only every grant.
Appr	Approval-bearing: segregation of duties (A1) applies and an initiator field is mandatory.
Initiator field	The field naming who initiated the item, resolved per §4.1.1.
Pos	<code>positionGrantable</code> — may appear in a Position's policy list.
Del	<code>delegationAllowed</code> — a delegate may grant it, bounded by the ceiling.
SA	<code>superAdminOnly</code> — only Super Admin may grant it.

Registry-wide invariants, verified at startup:

#	Invariant
RG-1	<code>Del = Y</code> requires <code>Sens = .</code> . A sensitive action is never delegable — a delegate may hold it, but only Super Admin may hand it out.
RG-2	<code>SA = Y</code> implies <code>Del = .</code> (GP-2).
RG-3	<code>Pos = .</code> implies <code>Del = .</code> and <code>SA = Y</code> (GP-1). Two actions qualify: <code>notepad:view-all</code> and <code>billing:set-terms</code> .
RG-4	<code>Appr = Y</code> requires a non-null initiator field (GP-5).

#	Invariant
RG-5	Every action names a resource or is explicitly <code>—</code> . There is no unspecified case.
RG-6	Every action has at least one API binding (§6.5).

Totals: 147 actions · 65 sensitive · 25 approval-bearing · 18 Super-Admin-only · 2 not position-grantable · 82 delegable · 38 people-domain · 101 business-domain · 8 derived-resource policies.

Foundation

Action	Resource	Domain	Sens	Appr	Initiator field	Pos	Del	SA
<code>identity:manage-geofence</code>	<code>geofenceLocation</code>	people	Y	·	—	Y	·	·
<code>identity:unlock-account</code>	<code>user</code>	people	Y	·	—	Y	·	·
<code>org:view-structure</code>	<code>department</code>	business	·	·	—	Y	Y	·
<code>org:view-people</code>	<code>user</code>	people	·	·	—	Y	Y	·
<code>org:view-policies</code>	<code>position</code>	business	Y	·	—	Y	·	·
<code>org:manage-departments</code>	<code>department</code>	business	Y	·	—	Y	·	Y
<code>org:manage-teams</code>	<code>team</code>	business	Y	·	—	Y	·	·
<code>org:manage-positions</code>	<code>position</code>	business	Y	·	—	Y	·	Y
<code>org:manage-designations</code>	<code>designation</code>	business	·	·	—	Y	Y	·
<code>access:view</code>	<code>user</code>	people	·	·	—	Y	Y	·
<code>access:delegate</code>	<code>user</code>	people	Y	·	—	Y	·	Y
<code>access:request-role-change</code>	<code>roleChangeRequest</code>	business	·	·	—	Y	Y	·
<code>access:decide-role-change</code>	<code>roleChangeRequest</code>	business	Y	Y	<code>requestedBy</code>	Y	·	Y
<code>audit:view</code>	<code>auditEntry</code>	derived	Y	·	—	Y	·	·
<code>audit:export</code>	<code>auditEntry</code>	derived	Y	·	—	Y	·	Y
<code>audit:manage-holds</code>	<code>legalHold</code>	derived	Y	·	—	Y	·	Y
<code>system:manage-settings</code>	—	business	Y	·	—	Y	·	Y
<code>system:manage-thresholds</code>	—	business	Y	·	—	Y	·	Y
<code>system:manage-integrations</code>	—	business	Y	·	—	Y	·	Y
<code>system:manage-retention</code>	—	business	Y	·	—	Y	·	Y

People

Action	Resource	Domain	Sens	Appr	Initiator field	Pos	Del	SA
<code>users:view</code>	<code>user</code>	people	·	·	—	Y	Y	·
<code>users:manage</code>	<code>user</code>	people	Y	·	—	Y	·	Y
<code>onboarding:manage</code>	<code>onboardingWorkflow</code>	people	·	·	—	Y	Y	·
<code>attendance:view-live</code>	<code>userStatus</code>	people	·	·	—	Y	Y	·
<code>attendance:view</code>	<code>attendanceRecord</code>	people	·	·	—	Y	Y	·
<code>attendance:export</code>	<code>attendanceRecord</code>	people	·	·	—	Y	Y	·
<code>attendance:correct</code>	<code>attendanceCorrection</code>	people	Y	Y	<code>requestedBy</code>	Y	·	·
<code>attendance:request-correction</code>	<code>attendanceCorrection</code>	people	·	·	—	Y	Y	·
<code>breaks:manage-policy</code>	<code>breakPolicy</code>	people	Y	·	—	Y	·	·

Action	Resource	Domain	Sens	Appr	Initiator field	Pos	Del	SA
breaks:review-breach	breakBreach	people	Y	Y	userId	Y	·	·
breaks:view	breakBreach	people	·	·	—	Y	Y	·
shifts:view	shift	people	·	·	—	Y	Y	·
shifts:manage	shift	people	·	·	—	Y	Y	·
shifts:approve	shiftRequest	people	·	Y	requestedBy	Y	Y	·
biometric:manage	biometricDevice	people	Y	·	—	Y	·	·
leave:view	leaveRequest	people	·	·	—	Y	Y	·
leave:request	leaveRequest	people	·	·	—	Y	Y	·
leave:request-wfh	leaveRequest	people	·	·	—	Y	Y	·
leave:manage-wfh-standing	leaveRequest	people	·	·	—	Y	Y	·
leave:acknowledge	leaveRequest	people	·	Y	requestedBy	Y	Y	·
leave:decide	leaveRequest	people	Y	Y	requestedBy	Y	·	·
leave:manage-types	leaveType	people	·	·	—	Y	Y	·
holidays:view	holiday	people	·	·	—	Y	Y	·
holidays:manage	holiday	people	·	·	—	Y	Y	·
payroll:view	payslip	people	Y	·	—	Y	·	·
payroll:manage	payrollRun	people	Y	·	—	Y	·	Y
payroll:manage-config	payrollConfig	people	Y	·	—	Y	·	Y
performance:view	performanceRecord	people	·	·	—	Y	Y	·
performance:view-aggregates	performanceRecord	people	·	·	—	Y	Y	·
performance:manage	performanceRecord	people	Y	Y	subjectId	Y	·	·

Sales

Action	Resource	Domain	Sens	Appr	Initiator field	Pos	Del	SA
territories:view	territory	business	·	·	—	Y	Y	·
territories:manage	territory	business	·	·	—	Y	Y	·
leads:view	lead	business	·	·	—	Y	Y	·
leads:create	lead	business	·	·	—	Y	Y	·
leads:edit	lead	business	·	·	—	Y	Y	·
leads:reassign	lead	business	·	·	—	Y	Y	·
callbacks:view	callback	business	·	·	—	Y	Y	·
callbacks:create	callback	business	·	·	—	Y	Y	·
callbacks:edit	callback	business	·	·	—	Y	Y	·
handovers:initiate	handover	business	·	·	—	Y	Y	·
handovers:receive	handover	business	·	·	—	Y	Y	·
handovers:record-disposition	handover	business	·	·	—	Y	Y	·
handovers:view	handover	business	·	·	—	Y	Y	·
deals:view	deal	business	·	·	—	Y	Y	·
deals:view-commercials	deal	business	Y	·	—	Y	·	·

Action	Resource	Domain	Sens	Appr	Initiator field	Pos	Del	SA
deals:create	deal	business	·	·	—	Y	Y	·
deals:edit	deal	business	·	·	—	Y	Y	·
deals:approve	deal	business	Y	Y	createdBy	Y	·	·
deals:allow-custom-terms	deal	business	Y	·	—	Y	·	·
deals:approve-contract	deal	business	Y	Y	createdBy	Y	·	·
deals:confirm-payment	deal	business	Y	Y	payment.initiatedBy	Y	·	·
deals:record-win	deal	business	Y	Y	closedBy	Y	·	·
deals:forecast	deal	business	·	·	—	Y	Y	·
approvals:decide	approvalRequest	business	Y	Y	requestedBy	Y	·	·
approvals:delegate	approvalDelegation	business	Y	·	—	Y	·	·

Delivery

Action	Resource	Domain	Sens	Appr	Initiator field	Pos	Del	SA
handoff:create	projectBrief	business	·	·	—	Y	Y	·
handoff:confirm	projectBrief	business	Y	Y	createdBy	Y	·	·
handoff:review	projectBrief	business	Y	Y	confirmedBy	Y	·	·
handoff:view	projectBrief	business	·	·	—	Y	Y	·
projects:view	project	business	·	·	—	Y	Y	·
projects:manage	project	business	·	·	—	Y	Y	·
projects:view-financials	project	business	Y	·	—	Y	·	·
tasks:view	task	business	·	·	—	Y	Y	·
tasks:assign	task	business	·	·	—	Y	Y	·
tasks:update	task	business	·	·	—	Y	Y	·
tasks:review	task	business	·	Y	assignedTo	Y	Y	·
tasks:manage-dependencies	task	business	·	·	—	Y	Y	·
tasks:log-time	timeEntry	business	·	·	—	Y	Y	·
resources:view	allocation	business	·	·	—	Y	Y	·
resources:allocate	allocation	business	·	·	—	Y	Y	·
resources:override-allocation	allocation	business	Y	·	—	Y	·	·
delivery:view	delivery	business	·	·	—	Y	Y	·
delivery:approve	delivery	business	Y	·	—	Y	·	·
delivery:share	delivery	business	·	·	—	Y	Y	·
delivery:signoff	delivery	business	Y	Y	deliveredBy	Y	·	·
changes:classify	changeRequest	business	Y	Y	raisedBy	Y	·	·
changes:assign	changeRequest	business	·	·	—	Y	Y	·

Finance

Action	Resource	Domain	Sens	Appr	Initiator field	Pos	Del	SA
billing:view-terms	billingTerms	business	Y	·	—	Y	·	·

Action	Resource	Domain	Sens	Appr	Initiator field	Pos	Del	SA
billing:set-terms	billingTerms	business	Y	·	—	·	·	Y
invoicing:view	invoice	business	·	·	—	Y	Y	·
invoicing:create	invoice	business	·	·	—	Y	Y	·
invoicing:issue	invoice	business	Y	Y	createdBy	Y	·	·
invoicing:send	invoice	business	·	·	—	Y	Y	·
invoicing:credit-note	creditNote	business	Y	·	—	Y	·	·
invoicing:manage-recurring	recurringSchedule	business	·	·	—	Y	Y	·
invoicing:manage-series	numberSeries	business	Y	·	—	Y	·	Y
payments:view	receipt	business	·	·	—	Y	Y	·
payments:record	receipt	business	·	·	—	Y	Y	·
payments:allocate	receipt	business	·	·	—	Y	Y	·
payments:refund	refund	business	Y	Y	requestedBy	Y	·	·
payments:reconcile	bankReconciliation	business	Y	Y	recordedBy	Y	·	·
receivables:view	invoice	business	·	·	—	Y	Y	·
receivables:dun	invoice	business	·	·	—	Y	Y	·
receivables:write-off	invoice	business	Y	Y	proposedBy	Y	·	·
payables:view	vendorBill	business	·	·	—	Y	Y	·
payables:create-bill	vendorBill	business	·	·	—	Y	Y	·
payables:approve-bill	vendorBill	business	Y	Y	createdBy	Y	·	·
payables:claim	expenseClaim	people	·	·	—	Y	Y	·
payables:approve-claim	expenseClaim	people	·	Y	claimedBy	Y	Y	·
payables:execute-run	paymentRun	business	Y	Y	approvedBy	Y	·	·
accounting:view-ledger	journalEntry	business	Y	·	—	Y	·	·
accounting:post-journal	journalEntry	business	Y	·	—	Y	·	·
accounting:manage-accounts	ledgerAccount	business	Y	·	—	Y	·	·
accounting:close-period	accountingPeriod	business	Y	Y	lastPostedBy	Y	·	·
accounting:reopen-period	accountingPeriod	business	Y	·	—	Y	·	Y
accounting:view-statements	—	business	Y	·	—	Y	·	·
accounting:tax-export	—	business	Y	·	—	Y	·	·

Client

Action	Resource	Domain	Sens	Appr	Initiator field	Pos	Del	SA
clients:view	client	business	·	·	—	Y	Y	·
clients:manage	client	business	Y	·	—	Y	·	·
clients:manage-requests	clientRequest	business	·	·	—	Y	Y	·
accounts:manage-ownership	account	business	Y	·	—	Y	·	·
accounts:view-revenue	account	business	Y	·	—	Y	·	·
renewals:view	renewalOpportunity	business	·	·	—	Y	Y	·

Cross-cutting

Action	Resource	Domain	Sens	Appr	Initiator field	Pos	Del	SA
communication:view	message	derived	·	·	—	Y	Y	·
communication:client-thread	message	derived	·	·	—	Y	Y	·
documents:view	document	derived	·	·	—	Y	Y	·
documents:upload	document	derived	·	·	—	Y	Y	·
documents:share-client	document	derived	Y	·	—	Y	·	·
documents:manage-templates	documentTemplate	business	·	·	—	Y	Y	·
reports:view	—	business	·	·	—	Y	Y	·
reports:build	—	business	·	·	—	Y	Y	·
reports:export	—	business	Y	·	—	Y	·	·
reports:view-financial	—	business	Y	·	—	Y	·	·
reports:view-lifecycle	—	business	Y	·	—	Y	·	Y
notices:manage	notice	business	·	·	—	Y	Y	·
notepad:view-all	notepad	people	Y	·	—	·	·	Y
sheets:manage	sheet	business	·	·	—	Y	Y	·

6.5 API Bindings

Every action bound to at least one HTTP route. **292 bindings**. A route registered at boot with no binding is a startup failure (RM-1); a binding naming an unregistered action fails the build (RM-2).

Paths are the design intent. **TECH.md** may refine them, but not the method-to-action mapping — that is the authorization contract, and changing it changes who can do what.

Method	Path	Action	Resource	Param
GET	/api/identity/geofences	identity:manage-geofence	geofenceLocation	—
POST	/api/identity/geofences	identity:manage-geofence	geofenceLocation	—
PATCH	/api/identity/geofences/:id	identity:manage-geofence	geofenceLocation	id
POST	/api/identity/geofences/:id/assign	identity:manage-geofence	geofenceLocation	id
POST	/api/identity/users/:id/unlock	identity:unlock-account	user	id
GET	/api/org/departments	org:view-structure	department	—
GET	/api/org/teams	org:view-structure	department	—
GET	/api/org/ladder/:departmentCode	org:view-structure	department	—
GET	/api/org/chart	org:view-people	user	—
GET	/api/org/positions/:id/holders	org:view-people	user	id
GET	/api/org/positions/:id/policies	org:view-policies	position	id
POST	/api/org/departments	org:manage-departments	department	—
PATCH	/api/org/departments/:id	org:manage-departments	department	id
POST	/api/org/teams	org:manage-teams	team	—
PATCH	/api/org/teams/:id	org:manage-teams	team	id
POST	/api/org/teams/:id/members	org:manage-teams	team	id
POST	/api/org/positions	org:manage-positions	position	—

Method	Path	Action	Resource	Param
PATCH	/api/org/positions/:id	org:manage-positions	position	id
PUT	/api/org/positions/:id/policies	org:manage-positions	position	id
GET	/api/org/designations	org:manage-designations	designation	—
POST	/api/org/designations	org:manage-designations	designation	—
GET	/api/access/effective/:userId	access:view	user	userId
GET	/api/access/who-can/:action	access:view	user	—
POST	/api/access/override	access:delegate	user	—
DELETE	/api/access/override/:id	access:delegate	user	id
POST	/api/access/role-change-request	access:request-role-change	roleChangeRequest	—
POST	/api/access/role-change-request/:id/decide	access:decide-role-change	roleChangeRequest	id
GET	/api/audit	audit:view	auditEntry	—
GET	/api/audit/:id	audit:view	auditEntry	id
POST	/api/audit/export	audit:export	auditEntry	—
GET	/api/audit/holds	audit:manage-holds	legalHold	—
POST	/api/audit/holds	audit:manage-holds	legalHold	—
DELETE	/api/audit/holds/:id	audit:manage-holds	legalHold	id
GET	/api/system/settings	system:manage-settings	—	—
PUT	/api/system/settings	system:manage-settings	—	—
PUT	/api/system/feature-flags	system:manage-settings	—	—
GET	/api/system/thresholds	system:manage-thresholds	—	—
PUT	/api/system/thresholds	system:manage-thresholds	—	—
GET	/api/system/integrations	system:manage-integrations	—	—
PUT	/api/system/integrations/:key	system:manage-integrations	—	key
GET	/api/system/retention	system:manage-retention	—	—
PUT	/api/system/retention	system:manage-retention	—	—
GET	/api/users	users:view	user	—
GET	/api/users/:id	users:view	user	id
POST	/api/users	users:manage	user	—
PATCH	/api/users/:id	users:manage	user	id
POST	/api/users/:id/status	users:manage	user	id
POST	/api/users/import	users:manage	user	—
GET	/api/onboarding	onboarding:manage	onboardingWorkflow	—
POST	/api/onboarding	onboarding:manage	onboardingWorkflow	—
POST	/api/onboarding/:id/steps/:stepId/complete	onboarding:manage	onboardingWorkflow	id
GET	/api/attendance/live	attendance:view-live	userStatus	—
GET	/api/attendance	attendance:view	attendanceRecord	—
GET	/api/attendance/:userId/:date	attendance:view	attendanceRecord	userId

Method	Path	Action	Resource	Param
POST	/api/attendance/export	attendance:export	attendanceRecord	—
POST	/api/attendance/corrections	attendance:correct	attendanceCorrection	—
POST	/api/attendance/corrections/:id/ approve	attendance:correct	attendanceCorrection	id
POST	/api/attendance/corrections/bulk	attendance:correct	attendanceCorrection	—
POST	/api/attendance/corrections/request	attendance:request- correction	attendanceCorrection	—
GET	/api/breaks/policies	breaks:manage-policy	breakPolicy	—
POST	/api/breaks/policies	breaks:manage-policy	breakPolicy	—
PATCH	/api/breaks/policies/:id	breaks:manage-policy	breakPolicy	id
POST	/api/breaks/policies/:id/preview	breaks:manage-policy	breakPolicy	id
POST	/api/breaks/policies/:id/assign	breaks:manage-policy	breakPolicy	id
POST	/api/breaks/breaches/:id/confirm	breaks:review-breach	breakBreach	id
POST	/api/breaks/breaches/:id/waive	breaks:review-breach	breakBreach	id
GET	/api/breaks/breaches	breaks:view	breakBreach	—
GET	/api/breaks/allowance/me	breaks:view	breakBreach	—
GET	/api/breaks/policies/resolve/:userId	breaks:view	breakBreach	userId
GET	/api/shifts	shifts:view	shift	—
GET	/api/shifts/assignments	shifts:view	shift	—
POST	/api/shifts	shifts:manage	shift	—
PATCH	/api/shifts/:id	shifts:manage	shift	id
POST	/api/shifts/assignments	shifts:manage	shift	—
POST	/api/shifts/requests/:id/decide	shifts:approve	shiftRequest	id
GET	/api/biometric/devices	biometric:manage	biometricDevice	—
POST	/api/biometric/devices	biometric:manage	biometricDevice	—
PATCH	/api/biometric/devices/:serial	biometric:manage	biometricDevice	serial
PUT	/api/biometric/mapping	biometric:manage	biometricDevice	—
GET	/api/biometric/punches	biometric:manage	biometricDevice	—
POST	/api/biometric/punches/replay	biometric:manage	biometricDevice	—
GET	/api/leaves	leave:view	leaveRequest	—
GET	/api/leaves/:id	leave:view	leaveRequest	id
GET	/api/leaves/balances/:userId	leave:view	leaveRequest	userId
GET	/api/leaves/calendar	leave:view	leaveRequest	—
POST	/api/leaves	leave:request	leaveRequest	—
DELETE	/api/leaves/:id	leave:request	leaveRequest	id
POST	/api/leaves/wfh	leave:request-wfh	leaveRequest	—
POST	/api/leaves/wfh/standing	leave:manage-wfh-standing	leaveRequest	—
DELETE	/api/leaves/wfh/standing/:id	leave:manage-wfh-standing	leaveRequest	id
POST	/api/leaves/:id/acknowledge	leave:acknowledge	leaveRequest	id

Method	Path	Action	Resource	Param
POST	/api/leaves/:id/decide	leave:decide	leaveRequest	id
GET	/api/leaves/types	leave:manage-types	leaveType	—
POST	/api/leaves/types	leave:manage-types	leaveType	—
PUT	/api/leaves/types/:id	leave:manage-types	leaveType	id
GET	/api/holidays	holidays:view	holiday	—
POST	/api/holidays	holidays:manage	holiday	—
PATCH	/api/holidays/:id	holidays:manage	holiday	id
GET	/api/payroll/payslips/mine	payroll:view	payslip	—
GET	/api/payroll/payslips/:id	payroll:view	payslip	id
GET	/api/payroll/runs	payroll:manage	payrollRun	—
POST	/api/payroll/runs	payroll:manage	payrollRun	—
PATCH	/api/payroll/runs/:id	payroll:manage	payrollRun	id
POST	/api/payroll/runs/:id/publish	payroll:manage	payrollRun	id
POST	/api/payroll/payslips/:id/revise	payroll:manage	payrollRun	id
GET	/api/payroll/structures	payroll:manage	payrollRun	—
PUT	/api/payroll/structures/:userId	payroll:manage	payrollRun	userId
GET	/api/payroll/config	payroll:manage-config	payrollConfig	—
PUT	/api/payroll/config	payroll:manage-config	payrollConfig	—
GET	/api/performance/:userId	performance:view	performanceRecord	userId
GET	/api/performance/:userId/kpis	performance:view-aggregates	performanceRecord	userId
POST	/api/performance/:userId/reviews	performance:manage	performanceRecord	userId
POST	/api/performance/cycles	performance:manage	performanceRecord	—
GET	/api/territories	territories:view	territory	—
POST	/api/territories	territories:manage	territory	—
PATCH	/api/territories/:id	territories:manage	territory	id
PUT	/api/territories/routing	territories:manage	territory	—
GET	/api/leads	leads:view	lead	—
GET	/api/leads/:id	leads:view	lead	id
GET	/api/leads/unrouted	leads:view	lead	—
POST	/api/leads	leads:create	lead	—
POST	/api/leads/import	leads:create	lead	—
PATCH	/api/leads/:id	leads:edit	lead	id
POST	/api/leads/:id/reassign	leads:reassign	lead	id
POST	/api/leads/bulk-reassign	leads:reassign	lead	—
GET	/api/callbacks	callbacks:view	callback	—
GET	/api/callbacks/:id	callbacks:view	callback	id
POST	/api/callbacks	callbacks:create	callback	—
PATCH	/api/callbacks/:id	callbacks:edit	callback	id

Method	Path	Action	Resource	Param
POST	/api/callbacks/:id/outcome	callbacks:edit	callback	id
POST	/api/handovers	handovers:initiate	handover	—
GET	/api/handovers/targets	handovers:initiate	handover	—
POST	/api/handovers/:id/accept	handovers:receive	handover	id
POST	/api/handovers/:id/decline	handovers:receive	handover	id
POST	/api/handovers/:id/disposition	handovers:record-disposition	handover	id
GET	/api/handovers	handovers:view	handover	—
GET	/api/handovers/:id	handovers:view	handover	id
GET	/api/deals	deals:view	deal	—
GET	/api/deals/:id	deals:view	deal	id
GET	/api/deals/:id/commercials	deals:view-commercials	deal	id
POST	/api/deals	deals:create	deal	—
PATCH	/api/deals/:id	deals:edit	deal	id
PATCH	/api/deals/:id/commercials	deals:edit	deal	id
POST	/api/deals/:id/approve	deals:approve	deal	id
POST	/api/deals/:id/custom-terms	deals:allow-custom-terms	deal	id
POST	/api/deals/:id/contract/approve	deals:approve-contract	deal	id
POST	/api/deals/:id/confirm-payment	deals:confirm-payment	deal	id
POST	/api/deals/:id/record-win	deals:record-win	deal	id
GET	/api/deals/wins-to-record	deals:record-win	deal	—
GET	/api/deals/forecast	deals:forecast	deal	—
GET	/api/approvals/queue	approvals:decide	approvalRequest	—
POST	/api/approvals/:id/decide	approvals:decide	approvalRequest	id
GET	/api/approvals/delegations	approvals:delegate	approvalDelegation	—
POST	/api/approvals/delegations	approvals:delegate	approvalDelegation	—
DELETE	/api/approvals/delegations/:id	approvals:delegate	approvalDelegation	id
POST	/api/handoff	handoff:create	projectBrief	—
POST	/api/handoff/:id/revise	handoff:create	projectBrief	id
GET	/api/handoff/pending-confirmation	handoff:confirm	projectBrief	—
POST	/api/handoff/:id/confirm	handoff:confirm	projectBrief	id
POST	/api/handoff/:id/query-back	handoff:confirm	projectBrief	id
GET	/api/handoff/feasibility-queue	handoff:review	projectBrief	—
POST	/api/handoff/:id/decide	handoff:review	projectBrief	id
GET	/api/handoff	handoff:view	projectBrief	—
GET	/api/handoff/:id	handoff:view	projectBrief	id
GET	/api/projects	projects:view	project	—
GET	/api/projects/:id	projects:view	project	id
POST	/api/projects	projects:manage	project	—

Method	Path	Action	Resource	Param
PATCH	/api/projects/:id	projects:manage	project	id
POST	/api/projects/:id/team	projects:manage	project	id
POST	/api/projects/:id/archive	projects:manage	project	id
GET	/api/projects/:id/profitability	projects:view-financials	project	id
GET	/api/tasks	tasks:view	task	—
GET	/api/tasks/:id	tasks:view	task	id
POST	/api/tasks	tasks:assign	task	—
POST	/api/tasks/:id/assign	tasks:assign	task	id
PATCH	/api/tasks/:id	tasks:update	task	id
POST	/api/tasks/:id/transition	tasks:update	task	id
POST	/api/tasks/:id/comments	tasks:update	task	id
POST	/api/tasks/:id/review	tasks:review	task	id
GET	/api/tasks/review-queue	tasks:review	task	—
POST	/api/tasks/:id/dependencies	tasks:manage-dependencies	task	id
DELETE	/api/tasks/:id/dependencies/:depId	tasks:manage-dependencies	task	id
POST	/api/tasks/:id/time	tasks:log-time	timeEntry	id
POST	/api/tasks/time/:entryId/approve	tasks:log-time	timeEntry	entryId
GET	/api/resources/capacity	resources:view	allocation	—
GET	/api/resources/workload	resources:view	allocation	—
POST	/api/resources/allocate	resources:allocate	allocation	—
POST	/api/resources/:assignmentId/override	resources:override-allocation	allocation	assignmentId
POST	/api/delivery/:projectId/deliver	delivery:approve	delivery	projectId
POST	/api/delivery/:id/share	delivery:share	delivery	id
GET	/api/delivery	delivery:share	delivery	—
GET	/api/delivery/:id	delivery:view	delivery	id
GET	/api/changes	delivery:view	delivery	—
POST	/api/delivery/:id/signoff	delivery:signoff	delivery	id
POST	/api/changes/:id/classify	changes:classify	changeRequest	id
GET	/api/changes	changes:classify	changeRequest	—
POST	/api/changes/:id/assign	changes:assign	changeRequest	id
GET	/api/billing/terms/:clientId	billing:view-terms	billingTerms	clientId
GET	/api/billing/rate-cards	billing:view-terms	billingTerms	—
PUT	/api/billing/terms/:clientId	billing:set-terms	billingTerms	clientId
POST	/api/billing/rate-cards	billing:set-terms	billingTerms	—
PATCH	/api/billing/rate-cards/:id	billing:set-terms	billingTerms	id
GET	/api/invoices	invoicing:view	invoice	—
GET	/api/invoices/:id	invoicing:view	invoice	id
POST	/api/invoices	invoicing:create	invoice	—

Method	Path	Action	Resource	Param
PATCH	/api/invoices/:id	invoicing:create	invoice	id
DELETE	/api/invoices/:id	invoicing:create	invoice	id
POST	/api/invoices/:id/issue	invoicing:issue	invoice	id
POST	/api/invoices/:id/send	invoicing:send	invoice	id
POST	/api/invoices/:id/credit-note	invoicing:credit-note	creditNote	id
GET	/api/invoices/recurring	invoicing:manage-recurring	recurringSchedule	—
POST	/api/invoices/recurring	invoicing:manage-recurring	recurringSchedule	—
PATCH	/api/invoices/recurring/:id	invoicing:manage-recurring	recurringSchedule	id
GET	/api/invoices/series	invoicing:manage-series	numberSeries	—
POST	/api/invoices/series	invoicing:manage-series	numberSeries	—
GET	/api/payments	payments:view	receipt	—
GET	/api/payments/:id	payments:view	receipt	id
POST	/api/payments	payments:record	receipt	—
POST	/api/payments/:id/allocate	payments:allocate	receipt	id
POST	/api/payments/:id/refund	payments:refund	refund	id
GET	/api/payments/reconciliation	payments:reconcile	bankReconciliation	—
POST	/api/payments/reconciliation/:id/ match	payments:reconcile	bankReconciliation	id
GET	/api/receivables/aging	receivables:view	invoice	—
GET	/api/receivables/statement/:clientId	receivables:view	invoice	clientId
GET	/api/receivables/collections	receivables:view	invoice	—
GET	/api/receivables/tds	receivables:view	invoice	—
POST	/api/receivables/:invoiceId/dun	receivables:dun	invoice	invoiceId
PUT	/api/receivables/dunning-schedule	receivables:dun	invoice	—
POST	/api/receivables/:invoiceId/write-off	receivables:write-off	invoice	invoiceId
GET	/api/payables/bills	payables:view	vendorBill	—
GET	/api/payables/bills/:id	payables:view	vendorBill	id
POST	/api/payables/bills	payables:create-bill	vendorBill	—
PATCH	/api/payables/bills/:id	payables:create-bill	vendorBill	id
POST	/api/payables/bills/:id/approve	payables:approve-bill	vendorBill	id
GET	/api/payables/claims/mine	payables:claim	expenseClaim	—
POST	/api/payables/claims	payables:claim	expenseClaim	—
POST	/api/payables/claims/:id/approve	payables:approve-claim	expenseClaim	id
GET	/api/payables/runs	payables:execute-run	paymentRun	—
POST	/api/payables/runs	payables:execute-run	paymentRun	—
POST	/api/payables/runs/:id/execute	payables:execute-run	paymentRun	id
GET	/api/accounting/ledger	accounting:view-ledger	journalEntry	—
GET	/api/accounting/trial-balance	accounting:view-ledger	journalEntry	—

Method	Path	Action	Resource	Param
GET	/api/accounting/journals	accounting:view-ledger	journalEntry	—
POST	/api/accounting/journals	accounting:post-journal	journalEntry	—
POST	/api/accounting/journals/:id/reverse	accounting:post-journal	journalEntry	id
GET	/api/accounting/accounts	accounting:manage-accounts	ledgerAccount	—
POST	/api/accounting/accounts	accounting:manage-accounts	ledgerAccount	—
PATCH	/api/accounting/accounts/:id	accounting:manage-accounts	ledgerAccount	id
GET	/api/accounting/periods	accounting:close-period	accountingPeriod	—
POST	/api/accounting/periods/:id/close	accounting:close-period	accountingPeriod	id
POST	/api/accounting/periods/:id/reopen	accounting:reopen-period	accountingPeriod	id
GET	/api/accounting/statements/pnl	accounting:view-statements	—	—
GET	/api/accounting/statements/balance-sheet	accounting:view-statements	—	—
GET	/api/accounting/statements/cash-flow	accounting:view-statements	—	—
POST	/api/accounting/tax/gstr1	accounting:tax-export	—	—
POST	/api/accounting/tax/gstr3b	accounting:tax-export	—	—
POST	/api/accounting/tax/tds	accounting:tax-export	—	—
GET	/api/clients	clients:view	client	—
GET	/api/clients/:id	clients:view	client	id
POST	/api/clients	clients:manage	client	—
PATCH	/api/clients/:id	clients:manage	client	id
POST	/api/clients/:id/credentials	clients:manage	client	id
DELETE	/api/clients/:id/credentials	clients:manage	client	id
GET	/api/clients/requests	clients:manage-requests	clientRequest	—
POST	/api/clients/requests/:id/respond	clients:manage-requests	clientRequest	id
POST	/api/accounts/:clientId/ownership	accounts:manage-ownership	account	clientId
GET	/api/accounts/:clientId/revenue	accounts:view-revenue	account	clientId
GET	/api/renewals	renewals:view	renewalOpportunity	—
GET	/api/communication/tracker	communication:view	message	—
GET	/api/projects/:id/messages/client	communication:client-thread	message	id
POST	/api/projects/:id/messages/client	communication:client-thread	message	id
GET	/api/documents	documents:view	document	—
GET	/api/documents/:id	documents:view	document	id
POST	/api/documents	documents:upload	document	—
POST	/api/documents/:id/versions	documents:upload	document	id
POST	/api/documents/:id/share	documents:share-client	document	id
GET	/api/documents/templates	documents:manage-templates	documentTemplate	—
POST	/api/documents/templates	documents:manage-templates	documentTemplate	—
GET	/api/reports	reports:view	—	—

Method	Path	Action	Resource	Param
GET	/api/reports/:key	reports:view	—	—
POST	/api/reports/custom	reports:build	—	—
POST	/api/reports/:key/export	reports:export	—	—
GET	/api/reports/financial/:key	reports:view-financial	—	—
GET	/api/reports/lifecycle	reports:view-lifecycle	—	—
GET	/api/notices	notices:manage	notice	—
POST	/api/notices	notices:manage	notice	—
PATCH	/api/notices/:id	notices:manage	notice	id
GET	/api/workspace/notepads	notepad:view-all	notepad	—
GET	/api/workspace/notepads/:userId	notepad:view-all	notepad	userId
GET	/api/workspace/sheets	sheets:manage	sheet	—
POST	/api/workspace/sheets	sheets:manage	sheet	—
PATCH	/api/workspace/sheets/:id	sheets:manage	sheet	id

6.6 CI Checks

#	Check	Failure mode
RM-9	Every action in PRD.md Appendix B appears in §6.4, and vice versa	Build fails
RM-10	Every action in §6.4 has at least one binding in §6.5	Build fails
RM-11	Every binding names an action defined in §6.4	Build fails
RM-12	Every registered Express route has a binding	Startup fails
RM-13	Invariants RG-1 to RG-6 hold for every row	Build fails
RM-14	Every binding whose action names a resource declares a param	Build fails
RM-15	Every Appr = Y action's resource policy returns a non-null initiatorField	Startup fails
RM-16	Every cell in the PRD.md §6 permission matrix maps to a registry action	Build fails

These are not aspirational. The registry is the source of truth precisely because nothing can diverge from it without the build going red.

7. Delegation Evaluation

Two distinct delegation mechanisms exist and must not be confused.

Mechanism	What it does	Constraints
Access delegation (<code>access:delegate</code>)	Editing another user's permission policies	Ceiling, boundary, seniority, root-of-trust — <code>PRD.md</code> §4.6
Approval delegation (<code>approvals:delegate</code>)	Temporarily routing an approval queue to someone else	The four conditions below

7.1 Approval Delegation — Four Conditions

Approval authority is not reducible to a numeric limit. All four must hold:

```
canDelegateApprovals(delegator, delegate, actionScope):

1. LIMIT
  delegate.approvalLimits[dimension] >= delegator.approvalLimits[dimension]
  evaluated per dimension: dealValueMax, discountPercentMax, allowCustomTerms

2. CAPABILITY
  delegate holds the underlying action with allowed = true
  → a Development Department Head with a high notional limit does not hold
  deals:approve and cannot receive a sales delegation

3. SCOPE
  delegate's scope for the action covers the delegator's record set
  → a Team Lead cannot delegate to a peer whose team scope excludes their
  pipeline

4. ORGANIZATIONAL BOUNDARY
  delegate.departmentRef == delegator.departmentRef
  OR delegate is an ancestor of delegator in the reporting chain
  → cross-department delegation of commercial authority is refused
```

Rules

#	Rule
DG-1	The delegation form filters candidates to those satisfying all four, and states the failing condition for anyone excluded. A silently absent candidate generates a support request.
DG-2	A delegate cannot delegate onward. One hop only.
DG-3	Delegation activates automatically when the delegator has approved leave covering the period, so the business does not stall because someone forgot.
DG-4	Every delegated decision records both the delegate and the original approver.
DG-5	Absolute constraint A1 applies through delegation: a delegate cannot approve a request the delegate themselves raised, and cannot approve one the delegator raised.
DG-6	Delegation is time-bounded. An open-ended delegation is refused.

8. Permission Delivery to the Client

GET /api/me/permissions

```
{
  "accountType": "employee",
  "position": { "code": "sales-team-lead", "name": "Sales Team Lead",
    "organizationalLevel": 70 },
  "department": "sales",
  "team": { "code": "sales-north", "name": "North Team" },
  "policies": {
    "leads:view": { "allowed": true, "scope": "team" },
    "deals:approve": { "allowed": true, "scope": "team" },
    "payroll:view": { "allowed": true, "scope": "own" },
    "users:manage": { "allowed": false, "scope": "none" }
  },
  "overrides": ["leads:view"],
  "approvalLimits": { "dealValueMax": 500000, "discountPercentMax": 25,
    "allowCustomTerms": true },
  "protectedNotes": {
    "payroll:view": "MF-4 — module access only; payslips require payroll:manage"
  },
  "fieldRestrictions": {
    "deal": ["commercials"]
  },
  "subordinateCount": 14
}
```

#	Rule
PD-1	Cached for the session, invalidated by a permissions:changed event pushed to the affected user.
PD-2	A user whose permissions narrow mid-session re-renders immediately. In-flight requests are still authorized server-side and fail with 403 if they no longer qualify.
PD-3	protectedNotes lets the interface explain why a control is unavailable, satisfying PRD.md NF-13.

9. Audit Integration

Event	Stream	Recorded
Policy grant, revoke, expiry	Access	actor, target, action, before, after, reason
Position policy change	Access	actor, position, diff, affected user count
Delegation change	Access	actor, target, constraint checks passed
Sensitive read	Access	actor, resource type and id, action
Segregation-of-duties block	Access	principal, action, resource — a blocked self-approval is visible, not silent
Super Admin bypass	Access	principal, action, resource
Business state change	Activity	actor, resource, transition, reason where required

Both streams are append-only and hash-chained ([PRD.md](#) §8.4).

10. Testing Requirements

#	Requirement
TS-1	Every action in <code>PRD.md</code> Appendix B has a positive and a negative test.
TS-2	Every protected constraint A1-A4 and P1-P8 has a test attempting the prohibited action through the API , not through an internal function.
TS-3	Every resource policy has a property test asserting <code>filter</code> and <code>check</code> agree.
TS-4	A matrix test walks every position against every action, asserting the resolved scope matches <code>PRD.md</code> §6.
TS-5	A leakage test asserts no list endpoint returns a record failing <code>check</code> , and no count differs from the length of the corresponding filtered list.
TS-6	A manifest test asserts route and manifest bijection.
TS-7	A segregation test asserts self-approval is refused for every approval-bearing workflow and every account type including super-admin.
TS-8	A field-projection test asserts denied fields are absent from the serialized response, not null.
TS-9	An override test asserts an override's scope takes effect.
TS-9a	An expiry test sets <code>expiresAt</code> to one second in the past, does not run the cleanup job , and asserts the override no longer applies on the very next authorization call. A test that runs the cleanup job first proves the job works, not that expiry is enforced.
TS-9b	A cache test asserts a permission set cached before an expiry is not served after it.
TS-10	Authorization tests run against the API surface. A test that calls the engine directly proves the engine works, not that the endpoint uses it.
TS-11	A domain test asserts that saving a policy with <code>all-people</code> scope on a business-domain action is refused at validation, not at request time.
TS-12	A mandatory-field test asserts that a principal holding <code>all-people</code> on <code>employee-directory</code> receives no compensation fields, and that an override cannot widen it.
TS-13	A registry test asserts every action is reachable by at least one API binding, and every binding names a registered action.
TS-14	A delegation test asserts each of the four approval-delegation conditions independently rejects an otherwise-valid candidate.
TS-15	A service-account test asserts position policies are never consulted, expiry is enforced, and every absolute constraint still binds.
TS-16	A tenancy test asserts that a query constructed without <code>organizationId</code> is refused, and that no endpoint returns a record from another organization.
TS-17	A deal-state test asserts <code>lifecycleStatus = won</code> is unreachable while any of the four predicates in <code>PRD.md</code> §10.5 is unmet, testing each predicate in isolation. This tests the state machine , not the authorization engine — see §11.
TS-19	A grantability test asserts GP-1 and GP-2 fail the build when violated, and that an action with <code>positionGrantable = false</code> cannot be saved into any Position policy through the API.
TS-20	A credit-disclosure test asserts that with BD-9 disabled the originating agent's deal response contains no monetary field, and with BD-9 enabled contains exactly <code>creditedValue</code> and nothing else from <code>commercials</code> .
TS-22	A finance-authority test asserts <code>billing:set-terms</code> cannot be written into any Position policy, and that a Finance Manager principal is refused on every write endpoint for billing terms.
TS-23	A period-integrity test asserts every posting endpoint is refused for a closed period, including for a Super Admin principal, and that only Super Admin can reopen.
TS-24	A double-entry test asserts an unbalanced journal cannot be persisted, and that the trial balance sums to zero after a randomised batch of every automatic posting type.
TS-25	An invoice-immutability test asserts an issued invoice cannot be edited or deleted through any endpoint by any principal, and that correction is possible only by credit note.

#	Requirement
TS-21	An account-boundary test asserts that holding account ownership grants no field of Deal.commercials through any endpoint.
TS-18	A separation test asserts a principal who holds deals:record-win is still refused when the deal is ineligible, and a principal who lacks it is refused on an eligible deal. Two independent gates, two independent failures.

11. Authorization Is Not the Workflow State Machine

Two different questions are asked about every state-changing request, and they must be answered by two different components.

Question	Answered by	Example
May this principal perform this action on this record?	Authorization engine	Does this user hold <code>deals:record-win</code> with a scope reaching this deal, and does segregation of duties permit it?
Is this record eligible for this transition?	The module's state machine	Is approval satisfied, contract signed, advance payment confirmed, stage closed?

Both must pass. Neither substitutes for the other.

```
// Every state-changing handler
authorize(principal, "deals:record-win", deal)    // may you?
DealStateMachine.assertCanTransition(deal, "won") // may it?
DealStateMachine.transition(deal, "won", principal) // then do it
```

Why this separation matters. Eligibility rules are business logic that changes with policy — a company might later require two signed contracts, or drop the advance-payment requirement for retainer clients. Authorization rules are security logic that changes with org structure. Putting eligibility into the authorization engine would mean every commercial policy change is a change to the security-critical component, reviewed and tested as though it were one.

Rules

#	Rule
WF-1	The authorization engine never inspects a record's workflow state to decide permission, except through a declared <code>Constraint</code> (§2.4) where the permission genuinely depends on state — for example, an agent may read commercials only on a deal they closed.
WF-2	A state machine never inspects a principal's permissions to decide eligibility. It receives the actor for the audit trail, not for the decision.
WF-3	A 403 means "not you". A 422 means "not this record, not yet". The two must be distinguishable by the caller, because the remedies are completely different: ask for access, versus complete the missing step.
WF-4	Each state machine names the specific unmet predicate in its 422. "Cannot transition" without a reason produces a support ticket every time.
WF-5	Modules owning a state machine: <code>deals</code> , <code>approvals</code> , <code>handoff</code> , <code>delivery</code> , <code>tasks</code> , <code>leave</code> , <code>shifts</code> , <code>break-management</code> , <code>onboarding</code> , <code>handovers</code> , <code>callbacks</code> , <code>leads</code> . Each declares its states, its legal transitions, and the predicate guarding each one.